

用CLI建s3 ·

```
用CLI建s3 · block all public access = false · bucket
policy = {
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
bucket 底下要有兩個folder · 一個叫 Images · 一個叫
Audios
```



下面直接給你一整套 AWS CLI 指令流程，你可以整段貼到終端機（改掉你要的 bucket name / region 就好）。

假設：

- bucket name：testusage1124
- region：ap-northeast-1（東京，可改）

1 建立 S3 Bucket

bash

複製程式碼

```
aws s3api create-bucket \
  --bucket testusage1124 \
  --region ap-northeast-1 \
  --create-bucket-configuration LocationConstraint=ap-northeast-1
```

如果你用的是 us-east-1，要拿掉 `--create-bucket-configuration` 那一行。

2 關掉「Block all public access」

AWS CLI 是用 `put-public-access-block` 來設定，這邊設成全部允許（等同 Console 把 Block 全部取消勾選）：

bash

複製程式碼

```
aws s3api put-public-access-block \
  --bucket testusage1124 \
  --public-access-block-configuration \
    BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPu
```

3 設定 Bucket Policy（允許所有人讀取物件）

把你提供的 policy 存成一個檔案（例如 `bucket-policy.json`）：

`bucket-policy.json` 內容：

json

複製程式碼

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
```

套上去：

bash

複製程式碼

```
aws s3api put-bucket-policy \
  --bucket testusage1124 \
  --policy file://bucket-policy.json
```

4 建立兩個「資料夾」：Images & Audios

在 S3 裡其實沒有真正的 folder，是用物件 key 來假裝資料夾，所以我們放兩個空物件：

bash

複製程式碼

```
aws s3api put-object \  
  --bucket testusage1124 \  
  --key Images/
```

```
aws s3api put-object \  
  --bucket testusage1124 \  
  --key Audios/
```

跑完之後，你在 Console 裡會看到：

- Images/
- Audios/

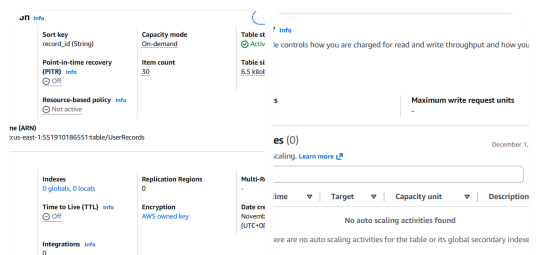
底下就可以開始上傳檔案，例如：

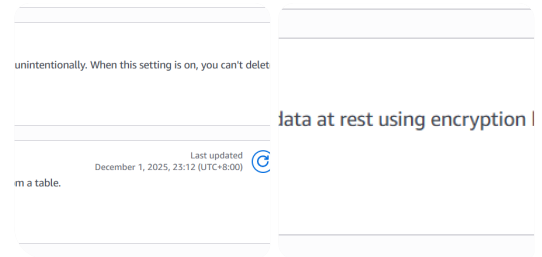
bash

複製程式碼

```
aws s3 cp myphoto.png s3://testusage1124/Images/myphoto.png  
aws s3 cp sound.mp3 s3://testusage1124/Audios/sound.mp3
```

如果你要再加上「靜態網站 hosting (index.html 那種)」也可以跟我說，我幫你把 CLI 後面那段一起整理 🍷





Dynamodb要有這些設定

已思考 36s >

好，照你貼的那些畫面，我幫你用 **AWS CLI** 重現一模一樣的 DynamoDB Table 設定。

目標條件整理一下：

- Table name : `UserRecords`
- Partition key : `user_id` (String)
- Sort key : `record_id` (String)
- Capacity mode : On-demand (`PAY_PER_REQUEST`)
- Indexes : 沒有 GSI / LSI
- PITR : Off (預設)
- Stream : Off (預設)
- TTL : Off (預設)
- Encryption : AWS owned key (預設)

1 用 CLI 建 table

bash

複製程式碼

```
aws dynamodb create-table \  
  --table-name UserRecords \  
  --attribute-definitions \  
    AttributeName=user_id,AttributeType=S \  
    AttributeName=record_id,AttributeType=S \  
  --key-schema \  
    AttributeName=user_id,KeyType=HASH \  
    AttributeName=record_id,KeyType=RANGE \  
  --billing-mode PAY_PER_REQUEST \  
  --region us-east-1
```

注意：

- “你現在的 ARN 看起來在 **us-east-1**，所以我加了 `--region us-east-1`，如果 CLI 預設就是這個 region 可以拿掉。”
- “`--billing-mode PAY_PER_REQUEST` 就是 On-demand，所以不要再加 `--provisioned-throughput`，加了會報錯。”

2 確認 table 設定正確

bash

複製程式碼

```
aws dynamodb describe-table \  
  --table-name UserRecords \  
  --region us-east-1 \  
  --query "Table.{TableName:TableName, \  
              BillingMode:BillingModeSummary.BillingMode, \  
              KeySchema:KeySchema, \  
              SSE: SSEDescription, \  
              StreamSpec:StreamSpecification}"
```

看重點：

- **BillingMode** 應該是 **PAY_PER_REQUEST**
- **KeySchema** 會顯示 **HASH=user_id**、**RANGE=record_id**
- **SSE** 如果沒特別設定，Encryption 會是 AWS owned key (Console 裡就會顯示你截圖那行)
- **StreamSpec** 會是 **StreamEnabled: false**

3 其他欄位 (PITR / TTL / Deletion protection)

這幾個在你需求裡都是 **Off**，CLI 不用特別下指令，預設就是：

- **PITR**：Point-in-time recovery 預設關閉
- **TTL**：預設關閉，要開才會用 `update-time-to-live`
- **Deletion protection**：預設 Off

所以只要跑完上面的 `create-table`，你的 table 設定就會跟截圖那個一樣了 🍌

如果你等一下還要我幫你用 CLI 建 **TTL** 或 **PITR** 打開，我可以再幫你補上對應指令。



怎麼看cli的region

可以，用幾個方式都看得到現在 AWS CLI 在用哪個 region 📌

1 最快的方法：`aws configure list`

```
bash
```

[複製程式碼](#)

```
aws configure list
```

會看到類似：

```
text
```

[複製程式碼](#)

name	value	type	location
----	-----	----	-----
profile	<not set>	...	
region	ap-northeast-1	config	~/.aws/config

這裡的 `region` 那一行就是目前 CLI 預設的 region。

2 直接問 region：`aws configure get region`

```
bash
```

[複製程式碼](#)

```
aws configure get region
```

終端機會直接只印出一行，例如：

```
text
```

[複製程式碼](#)

```
ap-northeast-1
```

3 看設定檔（如果你想確認檔案）

在 Windows 預設位置：

- C:\Users\你的帳號\.aws\config

內容大概像：

ini

 複製程式碼

```
[default]
region = ap-northeast-1
output = json
```

4 提醒：指令可以暫時覆蓋 region

就算 default 是別的，你也可以在某一個指令上加：

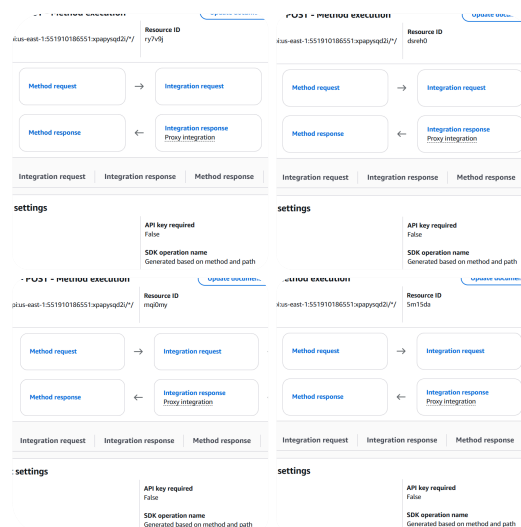
bash

 複製程式碼

```
aws dynamodb list-tables --region us-east-1
```

這一條就會用 `us-east-1`，不會改到全域設定。

你現在能跑一下 `aws configure list` 把輸出貼給我，我可以順便幫你確認一下你目前 CLI 到底是設在哪個 region。



創建apikey，操作的function是 lambda，叫 app-test2，要有四個resource

已思考 21s >

好，我幫你整理成「要達成作業條件」的一整套步驟：

- Lambda：app-test2（一個 function 處理四個 path）
- API Gateway：四個 resource + method
 - POST /generate-image
 - POST /generate-audio
 - POST /audio-callback
 - GET /history
- 這些 method 要走 Lambda Proxy integration
- 要啟用 API Key 保護

下面都用 **Console 點一點** 的做法（比 CLI 好操作很多）。

1 確認 Lambda integration 指到 app-test2

你截圖看起來已經是 Lambda integration 了，但再確認一下：

1. 到 **API Gateway** → 你的 **REST API**（ARN 裡看得出來是 us-east-1）。
2. 左邊選 /generate-image → 選 **POST** → 進到你貼的那個 Method execution 畫面。
3. 點右邊那個 **Lambda integration** 圖示或上方的 **Integration request** 分頁。
4. 確認：
 - Integration type: **Lambda Function**
 - Lambda Region: **us-east-1**
 - Lambda Function：app-test2

如果不是，按 **Edit** 換成 app-test2，儲存。

對以下三個 method 也做一次：

- POST /generate-audio
- POST /audio-callback
- GET /history

2 建立 API Key

1. 在左邊選單拉到下面，找到 **API keys**。
2. 按 **Create API key**

3. Name：隨便取，例如 `lab-key`
4. API key：選 **Auto generate**
5. 其他先預設 → 按 **Create**
6. 建好後，點進去這個 key，把 **API key value** 先複製起來（之後 Postman 要用）。

3 建立 Usage plan 並綁 API + Stage + Key

1. 左邊選單點 **Usage plans** → **Create**
2. Name：例如 `lab-usage-plan`
3. Throttling / Quota 可以先保持預設（或依作業要求調整）
4. 按 **Next**，在 **Associated API stages** 這邊：
 - 選你這個 REST API
 - stage 選你已經 deploy 的（例如 `prod` 或 `dev`）
 - 按 **Add**
5. 再按 **Next**，在 **API keys** 這步：
 - 點 **Add API key to usage plan**
 - 選剛剛建立的 `lab-key`
6. 完成後按 **Create**

這樣：

API key ↔ Usage plan ↔ API stage 三者就串好了。

4 讓四個 Method 都「需要 API Key」

現在你的截圖顯示 `API key required: False`，要改成 `True`：

對四個 method 逐一做：

1. 在 API Gateway 左邊選 `/generate-image` → **POST**
2. 在下方分頁選 **Method request**
3. 點右邊 **Edit**
4. 把 **API key required** 改成 `true` → 打勾 / 改成 **Yes**
5. 儲存
6. 回到 API 上方 **Actions** → **Deploy API**，選同一個 stage（例如 `prod`），重新 deploy。

同樣步驟套用到：

- **POST** `/generate-audio`

- POST `/audio-callback`
- GET `/history`

5 測試時要帶 `x-api-key` header

不管你用 Postman 還是前端：

- HTTP header 加上：

http

 複製程式碼

`x-api-key: <你剛剛複製的 API key value>`

只要 key 正確、Lambda 有回應，四個 endpoint 就會正常工作；如果 key 沒帶或錯誤，就會拿到 `403 Forbidden`。

如果你希望用 CLI 建 API key + usage plan 我也可以再幫你寫一套 CLI 指令版；或是你有 app-test2 的程式碼，我也可以幫你檢查要怎麼根據 `event["resource"]` / `event["path"]` / `event["httpMethod"]` 去分別處理四個 resource。

     ...

↳ policy file://bucket-policy.json

json檔要怎麼放上去

哦～你是問 `--policy file://bucket-policy.json` 那個 json 檔要怎麼準備對吧 🙌

其實就是「自己在電腦上先存一個文字檔」，再讓 CLI 去讀它而已，步驟給你兩種方式：

方法一：用記事本（最直覺）

以 Windows 為例：

1. 在桌面或你準備打指令的資料夾 空白處右鍵 → 新增 → 文字文件
2. 改名成：`bucket-policy.json`（副檔名要是 `.json`）
3. 右鍵 → 編輯（用記事本開）
4. 貼上你的 Policy 內容，例如：

json

 複製程式碼

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
```

5. 存檔，關掉記事本。

6. 在 **同一個資料夾** 開啟 PowerShell / CMD，然後執行：

bash

 複製程式碼

```
aws s3api put-bucket-policy ^
--bucket testusage1124 ^
--policy file://bucket-policy.json
```

(PowerShell 用  換行；直接打一行也可以)

方法二：直接在命令列建立檔案

PowerShell (一行建立)

在你要放檔案的資料夾裡執行：

powershell

 複製程式碼

```
@'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
```

```
"Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
'@ | Out-File -Encoding utf8 bucket-policy.json
```

跑完同樣在這個資料夾執行：

bash

 複製程式碼

```
aws s3api put-bucket-policy --bucket testusage1124 --policy file://bucket-policy.
```

路徑的小提醒

- `file://bucket-policy.json`：表示在「目前命令列所在的資料夾」。
- 如果檔案在別的地方，例如：`C:\policies\bucket-policy.json`，可以寫：

bash

 複製程式碼

```
--policy file://C:/policies/bucket-policy.json
```

(用 `/` 或 `\\` 都可以，重點不要漏掉 `file://`)

你可以先用**方法一**做一次，最不容易出錯。

如果你願意，把你現在打指令的資料夾路徑貼給我，我可以幫你寫成一模一樣可以貼上去用的版本。

     ...

我是要在aws的cloudshell執行

OK，在 **CloudShell** 裡放 `json` 檔更簡單，因為它本來就是一台小 Linux 主機。

你只要先在 CloudShell 裡「建立一個檔案」，再讓 `aws` 指令去讀就好。

下面給你兩種做法，你選一種就可以了👉

方法一：用 nano 編輯器建檔（最直覺）

在 CloudShell 視窗裡直接打：

```
bash
```

[複製程式碼](#)

```
nano bucket-policy.json
```

畫面會變成編輯器，接著把 policy 貼進去：

```
json
```

[複製程式碼](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
```

貼完之後：

1. 按 **Ctrl + O**（英文字母 O）→ 會問你檔名，直接按 Enter 存檔
2. 再按 **Ctrl + X** 退出 nano

回到指令模式後，在同一個目錄下執行：

```
bash
```

[複製程式碼](#)

```
aws s3api put-bucket-policy \
  --bucket testusage1124 \
  --policy file://bucket-policy.json
```

這樣就套上去了。

方法二：直接用 here-doc 建檔（不用開編輯器）

在 CloudShell 裡一次貼這段（注意 **EOF** 要一模一樣，前面不能有空白）：

```
bash
```

[複製程式碼](#)

```
cat << 'EOF' > bucket-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::testusage1124/*"
    }
  ]
}
EOF
```

這段會在目前資料夾建立一個 `bucket-policy.json` 檔。

接著一樣執行：

```
bash
```

[複製程式碼](#)

```
aws s3api put-bucket-policy \
  --bucket testusage1124 \
  --policy file://bucket-policy.json
```

小提醒

- 你可以用 `ls` 看一下，應該會看到 `bucket-policy.json` 在裡面。
- `file://bucket-policy.json` 代表「用目前資料夾的這個檔案」。
- 在 CloudShell 裡預設目錄通常是 `/home/cloudshell-user`，沒關係，只要你在那個位置建立檔案再下指令就可以了。

你先用其中一種方法試看看，如果 CLI 有跳錯誤訊息，把那行錯誤貼給我，我幫你一起看。

說明要等待適合用甚麼emoji

如果你想表達「請稍等一下」「處理中」，常用的有這些👉

1. ⌚ 沙漏未完成：正在處理中、還沒好
2. ⌚ 沙漏已完成：快好了 / 剛處理完
3. 🕒 / 🕒 時鐘：要花一點時間、請等候
4. 🙏 雙手合十：不好意思請稍等一下（比較有禮貌、客氣）
5. 😓 流汗笑臉：有點抱歉讓你久等，偏輕鬆口氣
6. 🔄 循環箭頭：刷新、排程中、正在重試 的感覺
7. 🌀 旋轉：類似「loading 中」

如果是正式一點的服務訊息，我會推薦：

- 「正在處理您的請求，請稍候 ⌚ 」
- 或是比較客氣的：「不好意思，可能需要一點時間處理，請稍等 🙏 ⌚ 」

你要偏可愛一點也可以用：「稍等我一下下喔 ⌚ 😓 」

📄 🍌 🗨️ ⬆️ ↺️ ...

python 等待300秒，怎麼寫

最簡單的寫法就是用 `time.sleep`：

python

📄 複製程式碼

```
import time

print("開始等待 300 秒...")
time.sleep(300) # 程式在這行暫停 300 秒 (5 分鐘)
print("等待結束")
```

如果你是在 `async` 函式裡（例如 FastAPI、async bot），就用 `asyncio.sleep`：

python

📄 複製程式碼

```
import asyncio

async def main():
    print("開始等待 300 秒...")
    await asyncio.sleep(300)
    print("等待結束")
```

📄 🍌 🗨️ ⬆️ ↺️ ...

```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history,
get_latest_record
import json
from urllib.parse import quote_plus
import asyncio

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)
s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
```

```
{HuggingFace_Token}"
    payload = {"inputs": prompt}

    response = requests.post(HuggingFace_URL,
                             headers=headers, json=payload, timeout=60)

    if response.status_code != 200:
        return jsonify({
            "success": False,
            "error": f"Hugging Face API 錯誤:
{response.status_code}",
            "details": response.text
        }), 500

    content_type = response.headers.get('Content-
Type', '')

    if 'application/json' in content_type:
        error_data = response.json()
        return jsonify({
            "success": False,
            "error": "Hugging Face 回傳錯誤",
            "details": error_data
        }), 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 存到 S3
    timestamp =
```

```
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    )
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
```

```
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')
        # 回傳網址(先訂好)
        timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    )
    filename = f"
```

```
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

if not prompt:
    print("❌ 缺少 prompt")
    return jsonify({"success": False, "error": "缺少 prompt"}), 400

base_callback = "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
callback_url = f"{base_callback}?user_id=
{encoded_user_id}&prompt=
{encoded_prompt}&s3_key={encoded_s3_key}"
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callBackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
```

```
"success": False,
"error": f"Suno API 錯誤:
{response.status_code}",
"details": response.text
}), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId ( Suno 回傳格式有時不同 )
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

await asyncio.sleep(100)
return jsonify({
    "download_url": get_latest_record(user_id),
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500
```

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 使用建立好的 s3_key
            s3_key = request.args.get("s3_key")

            try:
                s3.put_object(
                    Bucket=S3_BUCKET_NAME,
                    Key=s3_key,
                    Body=mp3,    # 直接用原始 bytes
                    ContentType="audio/mpeg"
                )
            except Exception as e:
                return jsonify({
                    "success": False,
                    "error": f"上傳 S3 失敗: {str(e)}"
                }), 500

            user_id = request.args.get("user_id", "no user id")
            prompt = request.args.get("prompt", "no prompt")
```

```
public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }, 500)

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})
```

```
    })

    except Exception as e:
        print("callback error:", e)
        return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
            "file_url": "... } , ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp
import requests
import asyncio

load_dotenv()
DC_token
="MTQzNTI3MDk4NDc4NDU0Nzg0Mg.Glnp2b.JOi1Ke
8hUGHyH_W0tp2q-D0WCtFiDglyUCZi3U"
# DC_token = os.getenv('DISCORD_TOKEN')
# FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')
```

```
FLASK_API_URL = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod"
```

```
handler = logging.FileHandler(filename='discord.log',  
encoding='utf-8', mode='w')  
intent = discord.Intents.default()  
intent.message_content = True  
intent.members = True
```

```
bot = commands.Bot(command_prefix='/',  
intents=intent)
```

```
@bot.event  
async def on_ready():  
    print(f"We are ready to go in, {bot.user.name}")
```

```
@bot.command()  
async def hello(ctx):  
    await ctx.send(f"Hello {ctx.author.mention}!")
```

```
@bot.command()  
async def create_photo(ctx, *, prompt: str = "未輸入  
prompt"):  
    try:  
        payload = {"prompt": prompt, "user_id":  
str(ctx.author.id)}  
        # response =  
requests.post("http://localhost:5000/generate-image",  
json=payload, timeout=60)  
        response = requests.post(f"  
{FLASK_API_URL}/generate-image", json=payload,  
timeout=60)  
        data = response.json()  
        """"  
  
        if not data.get("success"):  
            err = data.get("error") or "未知錯誤"  
            details = data.get("details")  
            msg = f"❌ Flask 回傳錯誤: {err}"  
            if details:  
                msg += f"\n 📄 詳細: {details}"  
            await ctx.send(msg)  
            return
```

```

"""
    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
        return

    file_url = data.get("download_url")
    if not file_url:
        await ctx.send("❌ Flask 沒有回傳檔案 URL")
        return

    file_resp = requests.get(file_url)
    if file_resp.status_code == 200:
        with io.BytesIO(file_resp.content) as f:
            await ctx.send(file=discord.File(f,
filename="generated.png"))
    else:
        await ctx.send("❌ 下載圖片失敗")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入
prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        response = requests.post(f"
{FLASK_API_URL}/generate-audio", json=payload,
timeout=10)
        wait_time = 300
        await ctx.send(f"⌚ Please wait for {wait_time}s,
Suno is making audio.")
        await asyncio.sleep(wait_time)
        data = response.json()
    """

    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('reply', '未知錯誤')}")

```

```
        return
        """

    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error') or data}")
        return

    # await ctx.send(f"data 內容: {data}")
    # 3 從 URL 下載檔案內容
    file_resp = data.get("download_url")
    # file_content = file_resp.text # 文字檔使用 text

    # 4 將內容回 Discord
    await ctx.send(f"✅ 從 Flask 取得音樂網址:\n{file_resp}")

    # task_id = data.get("task_id")
    # await ctx.send(f"✅ Suno 開始生成中，task_id: {task_id}")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        params = {"user_id": str(ctx.author.id)}
        response = requests.get(f"{FLASK_API_URL}/history", params = params,
                                timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"🗣️ 使用者 {ctx.author.id} 沒有任何紀錄。")
```

```

return

lines = []
for idx, r in enumerate(records, start=1):
    prompt = r.get("prompt", "(無prompt)")
    url = r.get("file_url", "(無網址)")
    dataType = r.get("type", "(無形別)")
    lines.append(f"{idx}. {dataType} : {prompt} → {url}")

msg = "\n".join(lines)

# 分段輸出避免超過 Discord 限制
if len(msg) > 1900:
    chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
    for chunk in chunks:
        await ctx.send(f"📖 你的紀錄:\n{chunk}", suppress_embeds=True)
    else:
        await ctx.send(f"📖 你的紀錄:\n{msg}", suppress_embeds=True)

except Exception as e:
    await ctx.send(f"❌ 查詢 DB 失敗: {e}")

bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)出現❌ Flask 回傳錯誤: {'message': 'Internal server error'}
```

已思考 35s >

關鍵問題有兩個：

1. 你在 **Flask** 的同步函式裡寫了 `await asyncio.sleep(100)`
2. 就算改成 `time.sleep(300)`，**Lambda + API Gateway** 也撐不了 300 秒，會被直接 timeout 掉。

所以才會在 Discord 那邊看到：

❌ Flask 回傳錯誤: {'message': 'Internal server error'}

這其實是 API Gateway 幫你包起來的 Lambda 500 / timeout。

一、為什麼現在一定會炸

你的 `/generate-audio` 現在是這樣（重點節錄）：

python

 複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    ...
    if not suno_request_id:
        ...
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500

    await asyncio.sleep(100)
    return jsonify({
        "download_url": get_latest_record(user_id),
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })
```

問題 1：await 在 def 裡不能用

`def generate_audio():` 是「同步函式」，裡面不能出現 `await`，Python 直接在載入檔案時就會 `SyntaxError`，所以 Lambda 根本跑不起來。

要用 `await` 必須是：

python

 複製程式碼

```
async def generate_audio():
    await asyncio.sleep(100)
```

但 Flask + awsgi 是 WSGI，同步框架，Lambda 那邊也不是 async server，這樣改也不適合。

問題 2：Lambda / API Gateway 無法等 300 秒

就算你改成：

python

 複製程式碼

```
time.sleep(300)
```

在 Lambda 也行不通：

- API Gateway 最大 timeout $\approx$ 29 秒
- Lambda 自己的 timeout 你大多也只設 30 秒左右
- 你要等 Suno + download + S3 + DB，再 sleep 300 秒，一定會超時 → API Gateway 回 `{"message": "Internal server error"}`

所以「讓 Lambda 幫你等 300 秒」這個設計本身就行不通。

二、正確做法：

「後端不等，Discord 自己等」

你現在的設計其實已經很接近對的方向了：

- `/generate-audio`：
 - 負責呼叫 Suno，給它 callback URL
 - 立刻回傳「已開始生成」+ `task_id`
- Suno 完成時呼叫 `/audio-callback`：
 - 下載 mp3、上傳 S3、寫 DynamoDB
- Discord `/create_audio`：
 - 呼叫 `/generate-audio`
 - 自己 `await asyncio.sleep(300)`
 - 再去查 DB (例如 `/history` 或一個 `/latest` API)，拿到最後一筆紀錄，送回使用者

我們改成這樣就好了 🙌

三、修改 Flask：`/generate-audio` 不要等待

✅ 把 `/generate-audio` 改成「送出 Suno 後就回」

替換原本的 `/generate-audio` 整段：

python

 複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
```

```
try:
    print("=== /generate-audio 被呼叫 ===")

    data = request.json
    print("收到的 JSON:", data)

    user_id = data.get('user_id', 'unknown')
    prompt = data.get('prompt')

    if not prompt:
        print("✗ 缺少 prompt")
        return jsonify({"success": False, "error": "缺少 prompt"}), 400

    # 預先決定好 S3 key, callback 用
    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
    s3_key = f"Audios/{filename}"

    encoded_prompt = quote_plus(prompt)
    encoded_user_id = quote_plus(str(user_id))
    encoded_s3_key = quote_plus(s3_key)

    base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/p"
    callback_url = (
        f"{base_callback}?user_id={encoded_user_id}"
        f"&prompt={encoded_prompt}"
        f"&s3_key={encoded_s3_key}"
    )

    payload = {
        "prompt": prompt,
        "style": "古典",
        "title": "AI Generated Music",
        "customMode": True,
        "instrumental": True,
        "model": "V3_5",
        "callbackUrl": callback_url
    }

    print("Suno payload:", payload)

    headers = {
        "Authorization": f"Bearer {Suno_Token}",
```

```
        "Content-Type": "application/json"
    }

    print("正在呼叫 Suno API...")
    response = requests.post(SUNO_URL, json=payload, headers=headers, timeout=10)

    print("Suno 回應狀態:", response.status_code)
    print("Suno 回應內容:", response.text)

    if response.status_code != 200:
        return jsonify({
            "success": False,
            "error": f"Suno API 錯誤: {response.status_code}",
            "details": response.text
        }), 500

    suno_resp = response.json()
    print("解析後 Suno JSON:", suno_resp)

    # 抓 taskId (盡量相容各種回傳格式)
    suno_request_id = (
        suno_resp.get("id")
        or suno_resp.get("requestId")
        or suno_resp.get("data", {}).get("taskId")
    )

    if not suno_request_id:
        print("✗ Suno 沒有回傳 taskId")
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500

    # ! 這裡「不要等待」，直接回成功 + task_id
    return jsonify({
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中，完成後會寫入 history",
        "s3_key": s3_key
    })

except Exception as e:
```

```
print("❌ Flask 在 /generate-audio 發生錯誤:", e)
return jsonify({"success": False, "error": str(e)}), 500
```

✅ 重點：

- “await asyncio.sleep(...) 完全移除”
- “也不再呼叫 get_latest_record”
- “只回傳：success, task_id, message, s3_key”

/audio-callback 那段你可以先不用動，它負責：

- Suno callback → 下載音檔
- 上傳 S3
- 寫入 DynamoDB insert_record

四、修改 Discord：由 bot 來等 300 秒 + 查歷史

✅ 改 /create_audio 指令

用 這個版本整段覆蓋 原本的 /create_audio：

python

📄 複製程式碼

```
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入prompt"):
    try:
        payload = {"prompt": prompt, "user_id": str(ctx.author.id)}

        # 1 叫 Flask 開始生成音樂 (這裡應該很快就回)
        response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload,
                                data = response.json())

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error') or data}")
            return

        task_id = data.get("task_id")
        wait_time = 300

        await ctx.send(
            f"⌚ Suno 正在生成音樂 (task_id: `{task_id}`) 。\n"
            f"請稍等 {wait_time} 秒，我幫你等完再查結果。 "
        )
```

```

# 2 bot 在這裡等 300 秒 (不影響 Lambda)
await asyncio.sleep(wait_time)

# 3 300 秒後，呼叫 /history 查最新紀錄
params = {"user_id": str(ctx.author.id)}
his_resp = requests.get(f"{FLASK_API_URL}/history", params=params, timeout=300)
his_data = his_resp.json()

if not his_data.get("success"):
    await ctx.send(f"❌ 查詢歷史紀錄失敗: {his_data.get('error', '未知錯誤')}")
    return

records = his_data.get("records", [])
if not records:
    await ctx.send("⚠️ 目前還查不到任何音樂紀錄，可能 Suno 還沒回來或 callback 延遲")
    return

latest = records[-1] # 最後一筆
url = latest.get("file_url", "(無網址)")
rec_type = latest.get("type", "(未知類別)")
rec_prompt = latest.get("prompt", "(無 prompt)")

await ctx.send(
    f"✅ 音樂生成完成！\n"
    f"類型: {rec_type}\n"
    f"Prompt: {rec_prompt}\n"
    f"網址: {url}"
)

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

```

這樣流程變成：

1. /create_audio → POST /generate-audio
2. Flask / Lambda 很快回 success + task_id (不會 timeout)
3. bot 在 Discord 端等 300 秒
4. 等完去 GET /history?user_id=...
5. 把最後一筆紀錄的 file_url 貼回來

你在程式裡這行：

 複製程式碼

這是 **真的 Discord Bot Token**，而且你已經貼出來了。

1. 到 [Discord Developer Portal](#) 把這個 Token 重新產生 (Reset Token)
2. 新 Token 放進 `.env` 或環境變數，用：

 複製程式碼

不然別人拿到 Token 就可以控制你的 bot。

如果你改成上面這個版本，再打 `/create_audio` 還是看到錯誤訊息，把 **Lambda CloudWatch log** 的那段錯誤貼給我，我可以幫你看是哪一行炸掉。



[https://testusage1124.s3.amazonaws.com/Audios/20251130120743_66ae21c6.mp3?
AWSAccessKeyId=ASIAYBADJDY3ZQ6AOSPB&Signatur
e=IU7co5rHQPvqtZtXSlgFEhdTUM%3D&x-amz-
security-
token=IQoJb3JpZ2luX2VjEB0aCXVzLWVhc3QtMSJHME
UCIQCZ%2BW9rmFZVQfJmGggjVlLygsdeHW2LmzXKs
wVAzmN2QGlgQZ%2B10HKgXeNHV8mC8orPJcvKkkkht
7fVEiyLCo%2Fd02lq7wlI5f%2F%2F%2F%2F%2F%2F%2
F%2F%2F%2FARABGgw1NTE5MTAxODY1NTEiDOikIOV
RGPuAxNDIXirDAj%2FdL1bbYglU05RmrwTGrqv%2FFxf
T0jwU1CWg14rTqkQ%2FhFLhww5loqd%2B2GISQRMet
MI5dhq5%2F7nAfK19u9GcTSjMgdz%2FP08lhNEOBfkh
YAAYkVldgC4tjNTJLwryYIsT9S8Emq8gaPNPurzbgujpee
TJw8Y7fsCfkSx22JXA2gSEKHui%2B2KrRb5CI%2Fi8VHH
xBYrhAjJnjliqHbV37aCtKbXjWbof36W8WY6gvj6UK%2F](https://testusage1124.s3.amazonaws.com/Audios/20251130120743_66ae21c6.mp3?AWSAccessKeyId=ASIAYBADJDY3ZQ6AOSPB&Signature=IU7co5rHQPvqtZtXSlgFEhdTUM%3D&x-amz-security-token=IQoJb3JpZ2luX2VjEB0aCXVzLWVhc3QtMSJHMEUCIQCZ%2BW9rmFZVQfJmGggjVlLygsdeHW2LmzXKswVAzmN2QGlgQZ%2B10HKgXeNHV8mC8orPJcvKkkkht7fVEiyLCo%2Fd02lq7wlI5f%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FARABGgw1NTE5MTAxODY1NTEiDOikIOVRGPuAxNDIXirDAj%2FdL1bbYglU05RmrwTGrqv%2FFxfT0jwU1CWg14rTqkQ%2FhFLhww5loqd%2B2GISQRMetMI5dhq5%2F7nAfK19u9GcTSjMgdz%2FP08lhNEOBfkhYAAYkVldgC4tjNTJLwryYIsT9S8Emq8gaPNPurzbgujpeeTJw8Y7fsCfkSx22JXA2gSEKHui%2B2KrRb5CI%2Fi8VHHxBYrhAjJnjliqHbV37aCtKbXjWbof36W8WY6gvj6UK%2F)

```
UM%2BEOlh9utjnVD0T%2BwHxfNgX4hH%2FJChdDS
x9qdlpLS9CdpqDeFDYDljnySV8kJ0k91os38WrQJ7k8y8
LuPZoCPrLHrkvcgU9LDJBWm9%2BxwENOijpnx9nfAzm
ZetKMgvoS0SkNWY%2BpFjzxE18XWJ%2BUZsRNRfQsg
NlgVRQj9Clcwo0BFOnrgBPRUnqOq3RUs65VVj96MM
%2FlsMkGO4BuL6Wij84MeD%2BSohcCjoQ%2F%2B8
OWA46v1V4NvQnNNclRMp0V7OdA4%2BeivYbIHKGsX
3HUdMl6xe%2Bf7UDDsmy2HlxTFCtmCn70nMmQGoy
ARuE6d9kwR3oQtX11uFdsQH2t5KEbdCCpge1vbcx5hn
AtZAhpIGWJFJR7VWuSBYJF%2BAaRhxhu%2B3JM6kyM
z49JXBWdj42vWf%2Bpy3WPKn%2FfeyQuhg%3D&Expi
res=1764508063
```

python 我要把第一個?之後的都剪掉

用字串處理就可以了，你要「第一個 ? 之前」的部分：

python

複製程式碼

```
url = "https://testusage1124.s3.amazonaws.com/Audios/20251130120743_66ae21c6.mp3?"

base_url = url.split('?', 1)[0]
print(base_url)
# 輸出: https://testusage1124.s3.amazonaws.com/Audios/20251130120743_66ae21c6.mp3
```

說明：

- `split('?', 1)` 只切一次，變成 `["前面沒參數的部分", "後面整串 query"]`
- 取 `[0]` 就是你要的「剪掉第一個 ? 之後全部」的網址。

如果你比較想用標準庫，也可以用 `urllib`：

python

複製程式碼

```
from urllib.parse import urlsplit, urlunsplit

parts = urlsplit(url)
base_url = urlunsplit((parts.scheme, parts.netloc, parts.path, '', ''))
print(base_url)
```



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history,
get_latest_record
import json
from urllib.parse import quote_plus
import asyncio

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)
s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
```

```
{HuggingFace_Token}"
    payload = {"inputs": prompt}

    response = requests.post(HuggingFace_URL,
                             headers=headers, json=payload, timeout=60)

    if response.status_code != 200:
        return jsonify({
            "success": False,
            "error": f"Hugging Face API 錯誤:
{response.status_code}",
            "details": response.text
        }), 500

    content_type = response.headers.get('Content-
Type', '')

    if 'application/json' in content_type:
        error_data = response.json()
        return jsonify({
            "success": False,
            "error": "Hugging Face 回傳錯誤",
            "details": error_data
        }), 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 存到 S3
    timestamp =
```

```
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    )
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
```

```
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')
        # 回傳網址(先訂好)
        timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    )
    filename = f"
```

```
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

if not prompt:
    print("❌ 缺少 prompt")
    return jsonify({"success": False, "error": "缺少 prompt"}), 400

base_callback = "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
callback_url = f"{base_callback}?user_id=
{encoded_user_id}&prompt=
{encoded_prompt}&s3_key={encoded_s3_key}"
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callBackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
```

```
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId ( Suno 回傳格式有時不同 )
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)

front_url = presigned_url.split('?', 1)[0]

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

return jsonify({
    "download_url": front_url,
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
```

```
return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 使用建立好的 s3_key
            s3_key = request.args.get("s3_key")

            try:
                s3.put_object(
                    Bucket=S3_BUCKET_NAME,
                    Key=s3_key,
                    Body=mp3,    # 直接用原始 bytes
                    ContentType="audio/mpeg"
                )
            except Exception as e:
                return jsonify({
                    "success": False,
                    "error": f"上傳 S3 失敗: {str(e)}"
                }), 500

            user_id = request.args.get("user_id", "no user id")
            prompt = request.args.get("prompt", "no prompt")
```

```
public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }, 500)

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    presigned URL
})
```

```
"reply": "Flask 收到音樂prompt"
}))

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
            "file_url": "... }], ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

❌ Flask 回傳錯誤: 'NoneType' object has no attribute
'get'
```

已思考 38s >

那個 ❌ Flask 回傳錯誤: 'NoneType' object has no attribute 'get' 很明顯就是：
在 Flask 裡哪裡有人對 None 呼叫 .get() 。

你這支程式裡最可疑的就是這兩行：

python

📄 複製程式碼

```
data = request.json
user_id = data.get('user_id', 'unknown')
```

```
prompt = data.get('prompt')
```

只要 `request.json` 解析失敗 / 沒有 JSON，就會變成 `None`，然後 `data.get(...)` 就噴：
`'NoneType' object has no attribute 'get'`

所以錯是在 **Flask** 的 `/generate-audio` 裡，不是在 Discord 那端。

1 先把 `request.json` 寫成安全版

建議你把三個地方都改一下（`/generate-image`、`/generate-audio`、`/audio-callback`）。

但你現在炸的是 `/generate-audio`，我先針對這個改。

✓ 把 `/generate-audio` 改成這樣（重點是前幾行）

python

複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        # 🟡 用 get_json(silent=True) 比較安全，不會直接丟 exception
        data = request.get_json(silent=True)
        print("收到的 JSON:", data)

        if not isinstance(data, dict):
            # 這時候 data 是 None 或 不是物件，直接回 400
            return jsonify({
                "success": False,
                "error": "請用 application/json 傳入正確的 JSON 物件"
            }), 400

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')

        if not prompt:
            print("✗ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        # 回傳網址(先訂好)
        timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
```

```
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/p
callback_url = f"{base_callback}?user_id={encoded_user_id}&prompt={encode

payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callBackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL, json=payload, headers=headers, timeout

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)
```

```

# ★ 抓 taskId (Suno 回傳格式有時不同)
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

# 先產一個 S3 URL (還沒真正有檔案，但你之前就是這樣用)
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)
front_url = presigned_url.split('?', 1)[0]

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

return jsonify({
    "download_url": front_url,
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

```

重點：

- 再也不會出現 `'NoneType' object has no attribute 'get'`，因為先檢查 `data` 有沒有內容。
- 如果真的沒傳 JSON，會變成：
`{"success": False, "error": "請用 application/json 傳入正確的 JSON 物件"}`

2 /generate-image 也一樣建議改一下

你的 /generate-image 也是：

```
python

data = request.json
prompt = data.get("prompt", "沒有收到圖片prompt")
```

複製程式碼

同樣風險，建議改成：

```
python

data = request.get_json(silent=True)
if not isinstance(data, dict):
    return jsonify({"success": False, "error": "請用 application/json 傳入正確的 JSON"})

prompt = data.get("prompt", "沒有收到圖片prompt")
user_id = data.get("user_id", "unknown")
```

複製程式碼

這樣就不會因為某次 API Gateway / Discord header 出問題就直接炸掉。

3 再來檢查 Discord 那邊

你在 Discord 那段是這樣處理錯誤的（之前你貼過）：

```
python

data = response.json()

if not data.get("success"):
    await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error') or data}")
    return
```

複製程式碼

現在 Flask 在 except 裡會回：

```
python

return jsonify({"success": False, "error": str(e)}), 500
```

複製程式碼

所以當 Flask 報錯，它會把 `'NoneType' object has no attribute 'get'` 塞進 `error` 給你，你那邊顯示成：

❌ Flask 回傳錯誤: 'NoneType' object has no attribute 'get'

這就是為什麼你在 Discord 看到這句。

改完 Flask 之後，如果再出錯，你看到的錯誤字串就會是比較乾淨的：

- 請用 `application/json` 傳入正確的 JSON 物件
- 或 Suno 未回傳 `taskId`
- 或其他你印出來的訊息。

4 小提醒：Presigned URL & front_url

現在 `/generate-audio` 直接回：

python

複製程式碼

```
front_url = presigned_url.split('?', 1)[0]
"download_url": front_url
```

注意兩件事：

1. 這個 URL 一開始點會 404，因為檔案還沒 upload (要等 Suno callback + `audio-callback` 路由跑完)。
2. 你的 Discord `/create_audio` 還是有 `await asyncio.sleep(wait_time)`，這樣用法 OK，就是讓 bot 等一段時間再把網址貼出來。

之後如果你發現「等 300 秒還是 404」，那就去 CloudWatch 看 `audio-callback` 有沒有被 Suno 叫到、DynamoDB 有沒有寫到，那又是下一個 debug 題目 XD

你先把 `/generate-audio` 前面那幾行改成 `request.get_json(silent=True)` + 判斷 `data`，

再試一次 `/create_audio`，如果還有新的錯誤訊息，把 **Flask log** 那段完整錯誤字串 (不是 Discord 只顯示一行的) 貼給我，我幫你繼續往下拆。

📄 🍌 🗨️ 🔗 ↺ ⋮

❌ Flask 回傳錯誤: 'NoneType' object has no attribute 'get'
from flask import Flask, request, jsonify,

```
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history,
get_latest_record
import json
from urllib.parse import quote_plus
import asyncio

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)
s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}
```

```
response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
filename = f"
```

```
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
```

```
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

    return jsonify({
        "success": True,
        "download_url": presigned_url, # 👉 這是 S3
presigned URL
        "reply": f"Flask 收到圖片prompt: {prompt}"
    })

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')
        # 回傳網址(先訂好)
        timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
        filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
        s3_key = f"Audios/{filename}"
```

```
encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

if not prompt:
    print("❌ 缺少 prompt")
    return jsonify({"success": False, "error": "缺少 prompt"}), 400

base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback"
callback_url = f"{base_callback}?user_id={encoded_user_id}&prompt={encoded_prompt}&s3_key={encoded_s3_key}"
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callBackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
                        json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤: {response.status_code}"
    })
```

```
"details": response.text
    }}, 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId (Suno 回傳格式有時不同)
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

return jsonify({
    "download_url": presigned_url,
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
```

```
try:
    print("=== 收到 Suno callback ===")
    data = request.json
    suno_items = data["data"]["data"]

    for item in suno_items:
        audio_url = item.get("source_audio_url")
        music_id = item["id"]

        if not audio_url:
            print("✖ 沒有 audio_url")
            return jsonify({
                "success": False,
                "error": "Suno 沒有回傳 audio_url"
            }), 500

        # 下載音檔
        mp3 = requests.get(audio_url).content

        # 使用建立好的 s3_key
        s3_key = request.args.get("s3_key")

        try:
            s3.put_object(
                Bucket=S3_BUCKET_NAME,
                Key=s3_key,
                Body=mp3,    # 直接用原始 bytes
                ContentType="audio/mpeg"
            )
        except Exception as e:
            return jsonify({
                "success": False,
                "error": f"上傳 S3 失敗: {str(e)}"
            }), 500

        user_id = request.args.get("user_id", "no user id")
        prompt = request.args.get("prompt", "no prompt")

        public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
        local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
```

```
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})

except Exception as e:
    print("callback error:", e)
```

```
        return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
            "file_url": "... } , ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp
import requests
import asyncio

load_dotenv()
DC_token
="MTQzNTI3MDk4NDc4NDU0Nzg0Mg.Glnp2b.JOi1Ke
8hUGHyH_W0tp2q-D0WCtFiDglyUCZi3U"
# DC_token = os.getenv('DISCORD_TOKEN')
# FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')
FLASK_API_URL = "https://xpapysqd2i.execute-api.us-
east-1.amazonaws.com/prod"
```

```
handler = logging.FileHandler(filename='discord.log',
encoding='utf-8', mode='w')
intent = discord.Intents.default()
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/',
intents=intent)

@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
    await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入
prompt"):
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        # response =
requests.post("http://localhost:5000/generate-image",
json=payload, timeout=60)
        response = requests.post(f"
{FLASK_API_URL}/generate-image", json=payload,
timeout=60)
        data = response.json()
        """
        if not data.get("success"):
            err = data.get("error") or "未知錯誤"
            details = data.get("details")
            msg = f"❌ Flask 回傳錯誤: {err}"
            if details:
                msg += f"\n 📄 詳細: {details}"
            await ctx.send(msg)
            return
        """
        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
```

```

{data.get('error', '未知錯誤'))
    return

    file_url = data.get("download_url")
    if not file_url:
        await ctx.send("❌ Flask 沒有回傳檔案 URL")
        return

    file_resp = requests.get(file_url)
    if file_resp.status_code == 200:
        with io.BytesIO(file_resp.content) as f:
            await ctx.send(file=discord.File(f,
            filename="generated.png"))
    else:
        await ctx.send("❌ 下載圖片失敗")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入 prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt, "user_id":
        str(ctx.author.id)}
        response = requests.post(f"
        {FLASK_API_URL}/generate-audio", json=payload,
        timeout=10)
        # wait_time = 300
        await ctx.send(f"⌚ Please wait for a moment,
        Suno is making audio.")
        # await asyncio.sleep(wait_time)
        data = response.json()
        """
        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
            {data.get('reply', '未知錯誤')}")
            return
        """

```

```
if not data.get("success"):
    await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error') or data}")
    return

# await ctx.send(f"data 內容: {data}")
# 3 從 URL 下載檔案內容
file_resp = data.get("download_url")
# front_url = file_resp.split('?', 1)[0]
# file_content = file_resp.text # 文字檔使用 text

# 4 將內容回 Discord
await ctx.send(f"✅ 從 Flask 取得音樂網
址:\n{file_resp}")

# task_id = data.get("task_id")
# await ctx.send(f"✅ Suno 開始生成中，task_id:
{task_id}")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        params = {"user_id":str(ctx.author.id)}
        response = requests.get(f"
{FLASK_API_URL}/history", params = params,
timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"🗿 使用者 {ctx.author.id} 沒有任
何紀錄。")
            return
```

```

lines = []
for idx, r in enumerate(records, start=1):
    prompt = r.get("prompt", "(無prompt)")
    url = r.get("file_url", "(無網址)")
    dataType = r.get("type", "(無形別)")
    lines.append(f"{idx}. {dataType} : {prompt} → {url}")

msg = "\n".join(lines)

# 分段輸出避免超過 Discord 限制
if len(msg) > 1900:
    chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
    for chunk in chunks:
        await ctx.send(f"📄 你的紀錄:\n{chunk}", suppress_embeds=True)
    else:
        await ctx.send(f"📄 你的紀錄:\n{msg}", suppress_embeds=True)

except Exception as e:
    await ctx.send(f"❌ 查詢 DB 失敗: {e}")

bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG) 甚麼錯誤

```

已思考 15s >

問題本體就是：

Flask 這兩行裡的 `data` 是 **None**，所以 `data.get(...)` 直接炸掉：

python

複製程式碼

```

data = request.json          # 這裡拿到的是 None
user_id = data.get('user_id', 'unknown')  # ← NoneType 沒有 get

```

一樣的寫法在 `/generate-image` 跟 `/generate-audio` 都有，所以只要有一次 `request.json == None`，

外層 `except` 會抓到，把錯誤字串丟回去，Discord 那邊就顯示：

❌ Flask 回傳錯誤: 'NoneType' object has no attribute 'get'

為什麼會是 `None` ？

Flask 裡的 `request.json` 只有在這些條件都有滿足時才會是 `dict`：

- HTTP method 有 body (POST/PUT...) ✓
- Content-Type 是 `application/json` ✓
- body 是合法的 JSON ✓

只要任一條件沒滿足，`request.json` 可能會是 `None`。

你這支用 `awscli` 接 API Gateway，有時 header / body 被轉出來的方式怪一點，就會踩到這種情況。

解法：用 `request.get_json(silent=True)` + 判斷

請把 `/generate-image` 跟 `/generate-audio` 的「前幾行」改掉。

我直接幫你改好整段，照貼就行。

✓ 修正版 `/generate-image`

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        # 改這裡
        data = request.get_json(silent=True)
        print("generate-image 收到的 JSON:", data)

        if not isinstance(data, dict):
            return jsonify({
                "success": False,
                "error": "請用 application/json 傳入正確的 JSON 物件"
            }), 400

        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}
```

```
response = requests.post(HuggingFace_URL, headers=headers, json=payload,

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

try:
    image = Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,
```

```
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={"Bucket": S3_BUCKET_NAME, "Key": s3_key},
        ExpiresIn=3600
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

```

    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

```

✓ 修正版 /generate-audio

python

 複製程式碼

```

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        # 📌 改這裡
        data = request.get_json(silent=True)
        print("收到的 JSON:", data)

        if not isinstance(data, dict):
            return jsonify({
                "success": False,
                "error": "請用 application/json 傳入正確的 JSON 物件"
            }), 400

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')

        if not prompt:
            print("✗ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        # 回傳網址(先訂好)
        timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
        filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
        s3_key = f"Audios/{filename}"

        encoded_prompt = quote_plus(prompt)
        encoded_user_id = quote_plus(str(user_id))
        encoded_s3_key = quote_plus(s3_key)

        base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/p

```

```
callback_url = f"{base_callback}?user_id={encoded_user_id}&prompt={encode

payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callBackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL, json=payload, headers=headers, timeout

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
```

```

        Params={"Bucket": S3_BUCKET_NAME, "Key": s3_key},
        ExpiresIn=3600
    )

    if not suno_request_id:
        print("✗ Suno 沒有回傳 taskId")
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500

    return jsonify({
        "download_url": presigned_url,
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

```

再提醒兩件事

1. Discord 那邊沒問題：

你用 `requests.post(..., json=payload)` 會自動帶 `Content-Type: application/json`，是正確的。

問題在 Flask 端沒檢查 `data` 就直接 `.get()`。

2. Discord Token 記得重置：

你現在程式裡有明文 `DC_token = "..."`，這個 token 建議立刻到 Discord Developer Portal reset，

然後改成用 `.env` 讀取，避免別人拿你的 bot 來玩。

你把這兩個 route 開頭改成 `request.get_json(silent=True)` + `if not isinstance(data, dict)` 之後，再試一次 `/create_audio` / `/create_photo`。

如果還有錯誤，CloudWatch / Lambda log 出來的 `print("收到的 JSON:", data)` 會顯示實際拿到的內容，到時你把那一行 log 貼給我，我可以一起看下一步。



generate_image是沒問題的

OK，那就鎖定 `generate_audio` 這隻來抓 bug。

現在這個錯誤：

✖ Flask 回傳錯誤: 'NoneType' object has no attribute 'get'

一定是這裡炸的：

python

複製程式碼

```
data = request.json
print("收到的 JSON:", data)

user_id = data.get('user_id', 'unknown')
prompt = data.get('prompt')
```

當 `request.json` 解析失敗時，`data` 會是 `None`，`None.get(...)` 就是這個錯誤。

`generate_image` 「剛好沒踩到」或是被 API Gateway 正確包成 JSON，所以沒事；
但 `generate_audio` 有時收到的不是 Flask 認得的 JSON，就會變 `None`。

只改 `generate_audio` 就好

把 原本的 `generate_audio` 函式完整替換成下面這版（`generate_image` 不要動）：

python

複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        # ✅ 這裡改用 get_json(silent=True)，並檢查是不是 dict
        data = request.get_json(silent=True)
        print("收到的 JSON:", data)

        if not isinstance(data, dict):
```

```
        return jsonify({
            "success": False,
            "error": "請用 application/json 傳入正確的 JSON 物件"
        }), 400

user_id = data.get('user_id', 'unknown')
prompt = data.get('prompt')

if not prompt:
    print("❌ 缺少 prompt")
    return jsonify({"success": False, "error": "缺少 prompt"}), 400

# 回傳網址(先訂好)
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/p"
callback_url = (
    f"{base_callback}"
    f"?user_id={encoded_user_id}"
    f"&prompt={encoded_prompt}"
    f"&s3_key={encoded_s3_key}"
)

payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callbackUrl": callback_url
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
```

```
        "Content-Type": "application/json"
    }

    print("正在呼叫 Suno API...")
    response = requests.post(SUNO_URL, json=payload, headers=headers, timeout=10)

    print("Suno 回應狀態:", response.status_code)
    print("Suno 回應內容:", response.text)

    if response.status_code != 200:
        return jsonify({
            "success": False,
            "error": f"Suno API 錯誤: {response.status_code}",
            "details": response.text
        }), 500

    suno_resp = response.json()
    print("解析後 Suno JSON:", suno_resp)

    # ★ 抓 taskId (Suno 回傳格式有時不同)
    suno_request_id = (
        suno_resp.get("id")
        or suno_resp.get("requestId")
        or suno_resp.get("data", {}).get("taskId")
    )

    # 先產一個 S3 的預先 URL (目前檔案可能還沒被 callback 寫入)
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600
    )

    if not suno_request_id:
        print("✗ Suno 沒有回傳 taskId")
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500
```

```
    return jsonify({
        "download_url": presigned_url,
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })

except Exception as e:
    print("❌ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500
```

關鍵只有：

python

 複製程式碼

```
data = request.get_json(silent=True)
if not isinstance(data, dict):
    return jsonify({...}), 400
```

這樣就算 API Gateway/Discord 某次傳來的 body 沒被 Flask 解析成 JSON，也不會再出現 `'NoneType' object has no attribute 'get'`，而是回一個乾淨的錯誤訊息。

你可以順便看一下 log

改完部署之後，如果又錯，CloudWatch 日誌裡會印出：

text

 複製程式碼

```
=== /generate-audio 被呼叫 ===
收到的 JSON: None
```

或是印出實際 data 長什麼樣。

如果你願意，把那一段「收到的 JSON: ...」貼給我，我可以幫你看是不是 API Gateway mapping 有怪。

     ...

```
def get_history(user_id: str):
```

```
# 查詢某個使用者的所有紀錄
resp = table.query(

KeyConditionExpression=Key("user_id").eq(user_id)
)
items = resp.get("Items", [])

# 只回傳記錄中的 prompt 跟 url
simple_items = [
    {
        "prompt": item.get("prompt"),
        "file_url": item.get("file_url"),
        "type": item.get("type")
    }
    for item in items
]
return simple_items
dynamodb按照時間排序，最新的是第一個
```

好，這邊分兩種情況跟你講清楚：

✅ 理想做法：用 sort key (時間) + Query 直接「由新到舊」

如果你的 DynamoDB table schema 是像這樣：

- Partition key：user_id
- Sort key：created_at (或 record_id，但內容是時間字串 / timestamp)

那只要在 query 時加上 ScanIndexForward=False，DynamoDB 就會用 sort key 由大到小回傳（也就是最新在最前面）。

你的 get_history 可以改成：

python

複製程式碼

```
from boto3.dynamodb.conditions import Key

def get_history(user_id: str):
    # 按 sort key 由新到舊排序
    resp = table.query(
        KeyConditionExpression=Key("user_id").eq(user_id),
        ScanIndexForward=False # 👉 False = 由大到小 (最新在前面)
```

```
)
items = resp.get("Items", [])

simple_items = [
    {
        "prompt": item.get("prompt"),
        "file_url": item.get("file_url"),
        "type": item.get("type")
    }
    for item in items
]
return simple_items
```

前提：你的 sort key 本身就代表「時間順序」，
例如 `created_at = "20251130120743"` 或是 Unix time (int) 。

😄 如果時間不是 sort key：只能程式裡排序

假設你的 table key schema 是：

- Partition key： `user_id`
- Sort key： `record_id` (但不是時間) ，
另外資料裡有一個欄位 `created_at` 存時間。

那就不能靠 DynamoDB 幫你排時間，只能拿到 items 後用 Python 排：

python

📄 複製程式碼

```
def get_history(user_id: str):
    resp = table.query(
        KeyConditionExpression=Key("user_id").eq(user_id)
    )
    items = resp.get("Items", [])

    # 假設每筆都有 created_at，例如 "20251130120743"
    items.sort(key=lambda x: x.get("created_at", ""), reverse=True)

    simple_items = [
        {
            "prompt": item.get("prompt"),
            "file_url": item.get("file_url"),
            "type": item.get("type")
        }
    ]
```

```

        for item in items
    ]
    return simple_items

```

建議你怎麼設計

既然你有「歷史紀錄」這種需求，會常常用：

- `user_id` 查
- 要「照時間排序」

最穩定的設計是：

- **Partition key** : `user_id`
- **Sort key** : `created_at` (或 `record_id` , 但值是時間 , 如 `20251202113045`)

這樣：

- `get_history` 只要加 `ScanIndexForward=False` 就能保證「最新第一個」。
- 之後如果你要做「查某個時間之後的紀錄」也可以直接在 `KeyConditionExpression` 用 `between` / `>=` 。



```

def get_history(user_id: str):
    """
    查詢某個使用者的所有紀錄，並依 created_at 排序
    """
    resp = table.query(
        KeyConditionExpression=Key("user_id").eq(user_id)
    )
    items = resp.get("Items", [])

    # 如果 created_at 是 ISO 格式字串，直接用字串排序即可
    items.sort(key=lambda x: x.get("created_at", ""),
               reverse=True)

    return items
只回傳前20筆

```

可以，直接在排序完之後切前 20 筆就好 🍷

你那段可以改成這樣：

python

 複製程式碼

```
from boto3.dynamodb.conditions import Key

def get_history(user_id: str):
    """
    查詢某個使用者的所有紀錄，依 created_at 由新到舊排序，
    並只回傳前 20 筆
    """
    resp = table.query(
        KeyConditionExpression=Key("user_id").eq(user_id)
    )
    items = resp.get("Items", [])

    # 依 created_at 排序 (新 → 舊)
    items.sort(key=lambda x: x.get("created_at", ""), reverse=True)

    # 只回傳前 20 筆
    return items[:20]
```

如果你之後還要再包成簡化版（只要 prompt / file_url / type），可以在 `return` 前再做一層 mapping，例如：

python

 複製程式碼

```
top_items = items[:20]
simple_items = [
    {
        "prompt": i.get("prompt"),
        "file_url": i.get("file_url"),
        "type": i.get("type"),
    }
    for i in top_items
]
return simple_items
```

